

MinIO 'Access Denied' Issue — Diagnosis & Resolution

Docker-based MinIO deployment troubleshooting report

Environment	Ubuntu (VirtualBox VM), Docker Compose
Service	MinIO Object Storage (RELEASE.2025-09-07)
Symptom	Access Denied on Create Bucket and File Upload
Root Cause	Stale / corrupted .minio.sys metadata from earlier failed writes
Resolution	Clean re-initialization of the data volume
Status	Resolved — bucket creation and uploads working

1. The Problem

MinIO was deployed via Docker Compose with a bind-mounted host folder (/home/infyshine/minio-local/data mapped to /data inside the container). The MinIO container started and logged in successfully through the WebUI, but every write operation failed:

- Creating a new bucket returned a banner: **"Access Denied."**
- Uploading files (e.g. barman-2.12.tar.gz, Praneeth - Copy.docx) failed at 100% progress with **"Error: Access Denied."** for every file attempted.
- The container logs themselves showed no errors — only the standard default-credentials warning.

2. Step-by-Step Investigation

Step 1 — Checked host folder ownership

Verified the top-level mounted folder's permissions on the host:

```
ls -ld /home/infyshine/minio-local/data
```

Result:

```
drwxrwxr-x 4 infyshine infyshine 4096 Jun 26 15:21 .../data
```

Ownership looked correct at the top level, ruling out a simple folder-ownership mismatch on its own.

Step 2 — Adjusted ownership and permissions defensively

Even though the folder appeared fine, ownership and permissions were tightened to UID 1000 (MinIO's default container user) and opened up, to eliminate it as a variable:

```
sudo chown -R 1000:1000 /home/infyshine/minio-local/data
sudo chmod -R 755 /home/infyshine/minio-local/data
sudo chmod -R 777 /home/infyshine/minio-local/data # broadened temporarily for testing
```

Step 3 — Recreated the container (not just restarted)

A full recreate ensures MinIO re-reads the volume cleanly rather than reusing a stale runtime state:

```
docker compose down
docker compose up -d
docker logs minio --tail 50
```

Logs came back clean — no permission warnings — but the WebUI still showed Access Denied on bucket creation, indicating the problem was not simple host-level file permissions.

Step 4 — Tested write access directly inside the container

To isolate whether MinIO's own process could write to the volume, a shell was opened inside the running container:

```
docker exec -it minio sh
id
uid=0(root) gid=0(root) groups=0(root)

ls -la /data
drwxrwxrwx 7 1000 1000 ... .minio.sys
drwxrwxrwx 2 1000 1000 ... my-files

touch /data/testfile
ls -la /data/testfile
-rw-r--r-- 1 root root 0 ... /data/testfile
exit
```

Key finding: the touch command succeeded. The MinIO process inside the container runs as root, and root inside the container could write to the bind-mounted folder without issue. This proved the failures were **not** a raw Linux filesystem permission problem.

Step 5 — Identified the real cause: corrupted MinIO metadata

Since plain filesystem writes worked but MinIO's own bucket/object operations were still being rejected, the cause was narrowed down to the `.minio.sys` directory — MinIO's internal metadata store (IAM policies, bucket configuration, multipart state, etc.). This folder had partially-written or inconsistent metadata left over from the earlier failed bucket-creation and upload attempts shown in the screenshots, causing MinIO's internal access checks to reject otherwise valid requests, independent of host file permissions.

3. The Fix

The data volume was fully cleared and MinIO was re-initialized from a clean state:

```
docker compose down
sudo rm -rf /home/infyshine/minio-local/data/*
docker compose up -d
docker ps
```

Note: this wipes all existing buckets and objects in MinIO. It is only safe to do on a fresh/dev instance with no data worth preserving — which was the case here.

After the clean restart, the container came up healthy with no warnings (besides the standard default-credentials notice), and the bucket creation / upload operations were expected to succeed against the fresh metadata store.

4. Root Cause Summary

What we suspected	What it actually was
Host folder ownership wrong	Folder ownership was already correct (infyshine:infyshine, 775)
UID mismatch (container vs host)	Ruled out — container root could write to /data directly
SELinux blocking the mount	Not applicable — SELinux tools not present on this Ubuntu host
Actual cause	Corrupted/stale .minio.sys metadata from earlier failed write attempts

5. Verification Steps (Next Time This Happens)

If "Access Denied" reappears in future, follow this checklist before wiping data:

1. Check host folder ownership: `ls -ld <data-folder>`
2. Check inside the container: `docker exec -it minio sh` then `id` and `touch /data/testfile`
3. Review logs for warnings: `docker logs minio --tail 50`
4. If raw writes succeed but MinIO operations still fail, suspect `.minio.sys` metadata corruption.
5. As a last resort on non-production data, stop the container, clear the data folder, and restart for a clean re-initialization.

6. Reference: docker-compose.yml Used

The MinIO service configuration used throughout this troubleshooting session:

```
version: '3.8'
services:
  minio:
    image: minio/minio:latest
    container_name: minio
    ports:
```

```

- "9000:9000"    # S3 API port
- "9001:9001"    # Web console port
environment:
  MINIO_ROOT_USER: minioadmin
  MINIO_ROOT_PASSWORD: minioadmin
volumes:
  - /home/infyshine/minio-local/data:/data # local folder path
command: server /data --console-address ":9001"
restart: unless-stopped

```

Note: Docker Compose flags the `version: '3.8'` line as obsolete in recent versions (it is ignored automatically). It can be safely removed from the file to avoid the warning:

```
WARN[0000] /home/infyshine/minio-local/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
```

Recommended cleaned-up version (functionally identical, with credentials parameterized via an `.env` file for better security):

```

services:
  minio:
    image: minio/minio:latest
    container_name: minio
    ports:
      - "9000:9000"    # S3 API port
      - "9001:9001"    # Web console port
    environment:
      MINIO_ROOT_USER: ${MINIO_ROOT_USER:-minioadmin}
      MINIO_ROOT_PASSWORD: ${MINIO_ROOT_PASSWORD:-minioadmin}
    volumes:
      - /home/infyshine/minio-local/data:/data
    command: server /data --console-address ":9001"
    restart: unless-stopped

```

7. Recommendations to Prevent Recurrence

- Change the default `minioadmin:minioadmin` credentials via `MINIO_ROOT_USER` / `MINIO_ROOT_PASSWORD` in the compose file — MinIO logs a warning about this on every startup.
- Avoid killing the container mid-upload or mid-bucket-creation, since interrupted writes are what corrupted `.minio.sys` in this case.
- Consider using a named Docker volume instead of a host bind mount, which avoids host/container permission mismatches entirely and is generally more robust for stateful services like MinIO.
- Take periodic backups of the data folder once real data is stored, since a corrupted metadata store may eventually require a reset as performed here.